Metaheuristics for Optimization

Series 2 : The Quadratic Assignment Problem

Part 1 : Return no later than September 29, 2025 (23h59)
Part 2 : Return no later than October 6, 2025 (23h59)

1 Quadratic Assignment Problem

The Quadratic Assignment Problem (QAP) is one of the fundamental combinatorial optimization problems, and it is important both in theory and practice. Intuitively, QAP can best be described as the problem of assigning a set of facilities (e.g. factories) to a set of locations (e.g. cities) with given distances between the locations and given flows (e.g. the amount of supplies transported) between the facilities. The goal then is to place all the facilities on different locations in such a way that the sum of the products between flows and distances is minimal. This problem is known to be NP-hard.

More formally, given n facilities and n locations together with two (symmetric) matrices $D = [d_{rs}]$ and $W = [w_{ij}]$ where d_{rs} is a distance between locations r and s and w_{ij} is the flow between facilities i and j, the QAP can be stated as follows :

$$\arg \min_{\psi \in S(n)} I(\psi) = \sum_{i,j=1}^{n} w_{ij} d_{\psi_i \psi_j}$$

where $S(n)$ is the set of all permutations of the integers $\{1, \ldots, n\}$; ψ_i gives the location of facility i in the current solution $\psi \in S(n)$.

2 Tabu Search

Tabu Search (TS) uses a local or neighborhood search procedure to iteratively move from a solution ψ to a solution ψ' in the neighborhood of ψ, until some stopping criterion has been satisfied.

The idea behind Tabu Search (TS) is to prevent the search from going backward. For instance, when moving from a solution $[2, 1, 0] \longrightarrow [2, 0, 1]$, the next step can't return to $[2, 1, 0]$. The swap between positions 1 and 2 is therefore considered *tabu* for a certain amount of time, called the *tabu tenure time*. The neighborhood list must not contain permutations of the current solution that are present in the tabu list.

3 Tabu Search for QAP

Fundamental concepts :

1. The *neighborhood* $N(\psi)$ of a given solution ψ is given by the classical 2-exchange which amounts to swapping two locations.

2. The *tabu list* contains the transpositions that have been tried during the last k iterations, k being the *tabu tenure time*.
3. The algorithm starts by randomly generating an initial assignment solution. At each iteration the best non tabu solution in $N^*(\psi)$ (the neighborhood of the current solution) is chosen as the new solution, even if it is worse than the current solution. The algorithm terminates after $t_{\max}$ iterations. Then the overall solution is the best assignment among the $t_{\max}$ iterations.
4. In this case, TS has one additional rule which introduces a type of *diversification mechanism* to the local search and which might be important to achieve good computational results in the long run : if a facility i has not been placed on a specific location r during the last u iterations, any move that does not place facility i on location r is forbidden. In fact, in such a situation, the algorithm forces to place a facility on one particular location. Fix $u = n^2$ (n being the number of locations and facilities).
5. **Optimizations and improvements :** TS is set so that, at each iteration, the best non-tabu solution within the neighbourhood is chosen as the new solution. This impleas that, given n cities and facilities, at each timestep the fitness for each neighbour need to be computed. The cost of one single fitness computation is of the order $O(n^2)$.

   Since the objective is to choose as new solution the one neighbor that performs the best within the neighborhood, one less computationally expensive approach would be to compute the difference of cost with the current solution. The $\Delta(\psi, i, j)$ obtained by exchanging locations ψ_i and ψ_j can be computed in $O(n)$ time using :

$$\Delta(\psi, i, j) = 2 \sum_{k \neq i,j} \big(w_{jk} - w_{ik}\big)\big(d_{\psi_i,\psi_k} - d_{\psi_j,\psi_k}\big)$$

   Finally, the permutation (i, j) for which $\Delta(\psi, i, j)$ is the smallest can be chosen (this is a minimization problem). The next fitness can be computed by adding the current fitness to $\Delta(\psi, i, j)$.

3.1 Part 1

The goal is understanding what a good problem is. Generate a problem with 12 cities and 12 facilities on a 100×100 grid. A solution for the problem with the Greedy Algorithm is already provided. Observe what happens for a problem generated randomly.

— Complete the functions `generate_cities_random`, `generate_weights_random`, and `compute_distance`.
— Randomly generate weights, ensuring that the weight matrix is symmetric and has only zeros on the diagonal.
— Compute the mean fitness of 1000 random permutations of size 10 (a random assignment of the facilities to the cities) and compare it to the fitness obtained using the greedy algorithm. What do you observe ?

— Compare the performance of the greedy algorithms and random permutations, for the given problem and the randomly generated problem. What do you observe ?
— *Optional :* Can you think at some ways of implementing a greedy algorithm ? How can you make a better problem than by generating weights and positions purely randomly ?

You can begin the implementaiton of the tabu search.

3.2 Part 2

Implement the tabu search with the diversification matrix and using the provided indications. Try with different tenure values and different tmax. What is the size of the neighborhood in the search space ?

4 Work to return

Submit code, results, and comments on Moodle :
— Monday, September 29, 2025 at 23h59 for Part 1
— Monday, October 7, 2025 at 23h59 for Part 2

Use Python notebooks to include code, results, and comments together. Graphs must include a title and axis legend when needed.

Test and discuss the effect of :
— varying the tenure time
— diversification mechanism
— (Optional) varying $t_{\max}$ (e.g. 500 to 20 000)